

AirFlux: Distributed Flight Operations & Pricing Engine

Complete Technical Architecture, Domain Model Breakdown, Inter-Service Dependency Contracts, and Operational Workflows for Airline Core, Flight Operations, and Pricing Microservices.

Project: AirFlux Management Platform	Architecture: Domain-Driven Microservices (Spring Boot 4.x / Java 17)
Core Modules: airlineCoreService, flightOpsService, pricingService	Data Isolation: Database-per-Service (MySQL 8.0)
Author: Mrityunjay Kumar & AirFlux Engineering	Security: Stateless JWT / Custom Headers (X-User-Id, X-Airline-Id)

Executive Architectural Summary: AirFlux is engineered to support high-volume commercial aviation operations by decoupling static master data from dynamic calendar operations and commercial revenue products. Each service maintains its own private MySQL database schema, preventing tight coupling and unbounded database queries. Operational coherence is achieved by decomposing flight management into three specialized abstraction tiers: **Flight** (the permanent route blueprint), **FlightSchedule** (the seasonal recurrence rule), and **FlightInstance** (the physical calendar flight that passengers book). Similarly, commercial revenue management is decoupled into **Fare** (the price, booking bucket, and amenity bundle) and **FareRules** (the contractual penalty policy for refunds and modifications).

1. System Architecture & Microservices Portfolio

AirFlux comprises five specialized business microservices, one shared core library, and planned cloud infrastructure:

Service	Port	Database	Domain & Primary Responsibility
common-lib	—	—	Shared DTOs, Enums (UserRole, FlightStatus, AirlineStatus), Embeddable benefit models, GlobalExceptionHandler.
userService	8082	user_db	Identity management, BCrypt hashing, JWT generation, and RBAC (ROLE_SYSTEM_ADMIN, ROLE_AIRLINE_OWNER, ROLE_USER).
locationService	8081	location_db	Global geographic catalog: Cities, Airports (IATA/ICAO), Timezones, nested Addresses, and GeoCoordinates.
airlineCoreService	8083	airline_db	Airline profile registration, admin status governance (approve/suspend/ban), aircraft fleet, and cabin seat configs.
flightOpsService	8084	flight_db	Route definitions (Flight), seasonal recurrence (FlightSchedule), automated daily FlightInstances, operational lifecycle state machine.
pricingService	8085	pricing_db	Dynamic ticket pricing, RBD booking classes, embedded multi-tier amenity perks, Baggage Policies, and legal Fare Rules.

2. Deep Dive: Flight vs. FlightSchedule vs. FlightInstance

A critical design challenge in airline systems is how to represent scheduled flights efficiently. If a route flies 3 times a week for 6 months, storing each departure as an independent record without hierarchy leads to massive data duplication, invalid schedules, and impossible route updates. AirFlux solves this with a clean 3-tier domain model:

Tier 1: Flight — The Route Blueprint

- **Class:** `com.airflux.flightOpsService.entity.Flight`
- **What it does:** Represents the static, ongoing authorization of a flight route. For example: *'British Airways flight BA101 operating from London Heathrow (LHR) to New York (JFK) using an Airbus A350'*.
- **What it does NOT have:** It has **no departure dates and no specific time of day**. It is an enduring organizational entity.
- **Fields:** `id`, `flightNumber` (unique across system), `airlineId`, `aircraftId`, `departureAirportId`, `arrivalAirportId`, `flightStatus` (default: SCHEDULED).

Tier 2: FlightSchedule — The Seasonal Timetable Rule

- **Class:** `com.airflux.flightOpsService.entity.FlightSchedule`
- **What it does:** Models the recurring seasonal timetable under which a Flight blueprint runs.
- **Fields:** `@ManyToOne Flight scheduledFlight`, `departureAirportId`, `arrivalAirportId`, `departureTime` (LocalTime, e.g. 10:00:00), `arrivalTime` (LocalTime, e.g. 14:30:00), `startDate` (e.g. 2026-06-01), `endDate` (e.g. 2026-08-31), `@ElementCollection List<DayOfWeek> operatingDays` (e.g. [MONDAY, WEDNESDAY, FRIDAY]), `isActive`.
- **Purpose:** Expresses recurring timetables without requiring staff to manually input individual flights for every day of the year.

Tier 3: FlightInstance — The Concrete Physical Departure

- **Class:** `com.airflux.flightOpsService.entity.FlightInstance`
- **What it does:** The real-world physical airplane departing on a **specific calendar date and time**. This is the entity travelers search for, reserve seats against, and that gate agents dispatch.
- **Fields:** `@ManyToOne Flight flight`, `scheduleId`, `departureDateTime` (LocalDateTime, e.g. 2026-06-05T10:00:00), `arrivalDateTime` (LocalDateTime, e.g. 2026-06-05T14:30:00), `totalSeats`, `availableSeats`, `flightInstanceStatus`, `minAdvanceBookingDays`, `maxAdvanceBookingDays`, `isActive`.
- **Operational State Machine:** Instances advance through distinct operational states: SCHEDULED → BOARDING → DEPARTED → IN_AIR → LANDED → ARRIVED → COMPLETED (or DELAYED, CANCELLED, DIVERTED).

Automated Flight Instance Generation Engine

```
// FlightScheduleServiceImpl.java – createFlightSchedule():
1. Verify Flight blueprint exists in repository.
2. Save FlightSchedule entity for the provided date range and operating days.
3. Automated Date Generator Loop:
   LocalDate startDate = savedFlightSchedule.getStartDate();
   LocalDate endDate = savedFlightSchedule.getEndDate();
   List<DayOfWeek> operatingDays = savedFlightSchedule.getOperatingDays();

   for (LocalDate date = startDate; !date.isAfter(endDate); date = date.plusDays(1)) {
       if (operatingDays.contains(date.getDayOfWeek())) {
           flightInstanceRequest.setDepartureDateTime(LocalDateTime.of(date,
savedFlightSchedule.getDepartureTime()));
           flightInstanceRequest.setArrivalDateTime(LocalDateTime.of(date,
savedFlightSchedule.getArrivalTime()));
           // Persists FlightInstance with initial availableSeats = aircraft totalSeats
           flightInstanceService.createFlightInstance(flightInstanceRequest, airlineId);
       }
   }
}
```

Why are they dependent on each other?

1. **FlightSchedule depends on Flight:** A schedule cannot exist in a vacuum; it must know what route, origin/destination airports, and operating airline it serves.
2. **FlightInstance depends on both Flight and FlightSchedule:** It needs Flight to inherit aircraft configuration, total seating capacity, and route endpoints. It needs scheduleId to maintain operational auditability back to the seasonal timetable that spawned it.

3. Deep Dive: Fare vs. FareRules vs. BaggagePolicy

In aviation revenue management, a ticket price is never just a simple scalar number. Airlines sell multiple branded commercial products on the exact same flight. For example, two passengers seated side-by-side in Economy may hold different fare products:

Fare Attribute	Economy Light / Saver	Economy Flex / Plus
Base Fare + Fees	\$150 total (\$110 base + \$30 tax + \$10 fee)	\$280 total (\$220 base + \$40 tax + \$20 fee)
RBD Code	'O' (Deep discount economy bucket)	'Y' (Full fare flexible economy bucket)
Passenger Amenities	No free meal, standard boarding, paid seat selection	Complimentary meals, priority boarding, advance seat selection
Refund Policy	Non-refundable (\$0 refund upon cancellation)	Refundable (100% refund up to 7 days before departure)
Change Penalty	\$100 date change fee	\$0 fee for date modifications up to 24 hours prior

A. Fare — The Commercial Product & Price

- **Class:** `com.airflux.pricingService.entity.Fare`
- **What it does:** Models the commercial package offered to travelers on a flight route.
- **Fields:** `flightId`, `cabinClassType` (ECONOMY, PREMIUM_ECONOMY, BUSINESS, FIRST), `rbdCode` (single-letter booking designator), `baseFare`, `taxesAndFees`, `airlineFees`, `currentPrice`, `fareLabel` (e.g. 'Best Value', 'Flexible').
- **Pricing Equation:** In `FareMapper.java`, price is computed dynamically: `currentPrice = baseFare + taxesAndFees + airlineFees`.
- **Embedded Value Objects:** AirFlux bundles five rich benefit objects into each Fare:
 1. **SeatBenefits:** `extraSeatSpace`, `preferredSeatChoice`, `advanceSeatSelection`, `guaranteedSeatTogether`.
 2. **BoardingBenefits:** `priorityBoarding`, `priorityCheckIn`, `fastTrackSecurity`.
 3. **InFlightBenefits:** `complimentaryMeals`, `premiumMealChoice`, `inflightInternet`, `inFlightEntertainment`, `complimentaryBeverages`.
 4. **FlexibilityBenefits:** `freeDateChange`, `partialRefund`, `fullRefund`.
 5. **PremiumServiceBenefits:** `loungeAccess`, `airportTransfer`.

B. FareRules — The Legal & Policy Contract

- **Class:** `com.airflux.pricingService.entity.FareRules`
- **What it does:** Defines the post-booking legal policy governing modifications and cancellations.
- **Fields:** `@OneToOne` Fare fare, `isRefundable`, `isChangeable`, `cancellationFee` (penalty deducted from refund), `changeFee` (charge to move date), `refundDeadlineDays` (must cancel X days before flight), `changeDeadlineHours` (cutoff window).

C. BaggagePolicy — Baggage Allowance Terms

- **Class:** `com.airflux.pricingService.entity.BaggagePolicy`
- **What it does:** Specifies weight and piece limits attached to a fare tier.
- **Fields:** `@OneToOne` Fare fare, `cabinBaggageMaxWeight`, `cabinBaggagePieces`, `checkInBaggageMaxWeight`, `freeCheckedBagsAllowance`, `priorityBaggage`, `extraBaggageAllowance`.

D. Dependency & Purpose Between Fare and FareRules

- **Relational Binding:** In `Fare.java`, the relationship is mapped as:
`@OneToOne(mappedBy = 'fare', cascade = CascadeType.ALL, orphanRemoval = true) private FareRules fareRules;`
- **Database Enforcement:** In `FareRulesServiceImpl.java`, a rule cannot be created unless its `fareId` already exists in the database. Moreover, an active fare is restricted to exactly one `FareRules` set (verified via `existsByFareId`).
- **Business Purpose:**
 1. **During Flight Search:** The UI queries Fare to display the ticket price and in-flight perks, while consulting FareRules to display policies (e.g. 'Non-refundable, \$75 change fee').
 2. **During Cancellation/Refund:** When a traveler cancels a booking, the system inspects the ticket's FareRules. If `isRefundable == false`, the refund is denied. If true, the net refund returned to the passenger is computed as:
 Net Refund = Total Paid - `cancellationFee`
 3. **Lifecycle Cleanup:** Cascade and orphan removal ensure that deleting a commercial fare tier automatically purges its obsolete legal rules and baggage allowances.

4. Deep Dive: Airline Core Service & Fleet Capacity

The **Airline Core Service** (:8083) acts as the carrier master database, enforcing airline business rules and fleet capacity constraints:

- **Carrier Ownership:** Each airline is owned by a verified user (ROLE_AIRLINE_OWNER) via ownerId. Headquarters location is mapped to headQuaterCityId from Location Service.
- **Platform Administrative Oversight:** An airline begins in PENDING status and cannot schedule commercial flights until a platform administrator approves it to ACTIVE (via /approve, /suspend, /ban).
- **Strict Seating Capacity Invariant:** In AircraftServiceImpl.java, when adding an aircraft (e.g. Boeing 787), the system enforces:
aircraft.getSeatingCapacity() >= (economySeats + premiumEconomySeats + businessSeats + firstClassSeats)
If the sum of individual cabin classes exceeds total physical airframe capacity, the operation throws an IllegalArgumentException.
- **Automated Maintenance Safeguards:** Aircraft entities track nextMaintenanceDate. The helper method requiresMaintenance() alerts flight dispatchers if routine maintenance is required within 14 days, preventing unsafe flight assignments.

5. Inter-Service Entity Relationship Matrix

Because AirFlux strictly adheres to Database-per-Service architecture, entity relationships across service boundaries are maintained via logical ID references rather than cross-database SQL foreign keys:

Source Entity	Reference Field	Target Service	Target Entity	Architectural & Domain Purpose
Airline	ownerId	userService	User.id	Links carrier account to authenticated airline manager.
Airline	headQuaterCityId	locationService	City.id	Resolves carrier corporate headquarters city & country.
Aircraft	currentAirportId	locationService	Airport.id	Tracks physical airport parking / current location of tail.
Flight	airlineId	airlineCoreService	Airline.id	Identifies operating commercial carrier for the route.
Flight	aircraftId	airlineCoreService	Aircraft.id	Assigns aircraft type, seating layout, and range capabilities.
Flight	departureAirportId	locationService	Airport.id	Origin terminal / IATA airport code.
Flight	arrivalAirportId	locationService	Airport.id	Destination terminal / IATA airport code.
FlightSchedule	scheduledFlight	flightOpsService	Flight.id	Links recurring timetable to master route blueprint.
FlightInstance	flight, scheduleId	flightOpsService	Flight & Schedule	Ties concrete departure to parent blueprint and timetable.
Fare	flightId	flightOpsService	Flight.id	Attaches commercial ticket pricing to a flight route.
FareRules	fare	pricingService	Fare.id	1:1 contractual policy for refunds and date modifications.
BaggagePolicy	fare	pricingService	Fare.id	1:1 baggage allowance weight and piece limits.

6. End-to-End Operational Lifecycle

Here is the step-by-step end-to-end execution workflow of the complete AirFlux platform:

Phase 1: User & Location Master Data Setup

1. Airline manager registers via POST /api/v1/auth/signup with role ROLE_AIRLINE_OWNER in User Service (:8082), obtaining a JWT token.
2. System administrator populates the global geographical database via Location Service (:8081) with cities (e.g. London, New York) and airports (e.g. LHR, JFK with timezones and coordinates).

Phase 2: Airline Onboarding & Fleet Configuration

1. Airline manager registers carrier profile via POST /api/v1/airlines (passing header X-User-Id) in Airline Core Service (:8083).
2. Platform administrator reviews and approves airline status to ACTIVE.
3. Airline owner registers aircraft fleet (e.g., Airbus A350, 331 seats) configuring Economy (219), Premium Economy (56), Business (56), and First Class (0) cabin capacities.

Phase 3: Route Creation & Automated Instance Generation

1. Airline owner creates route blueprint via POST `/api/v1/flights` in Flight Ops Service (:8084) linking LHR to JFK.
2. Owner submits seasonal schedule via POST `/api/v1/flight-schedules` (operating Mon, Wed, Fri at 10:00 AM from June 1 to August 31).
3. **System Automation:** The date generator loops through every date, automatically persisting concrete `FlightInstance` departures with initial 331 available seats.

Phase 4: Pricing, Amenity Bundling & Fare Rules

1. Airline owner creates tiered Fare offerings in Pricing Service (:8085) for that flight (e.g. 'Economy Saver' at \$150 vs 'Economy Flex' at \$280).
2. Owner configures embedded amenity bundles (meals, wifi, priority boarding).
3. Owner attaches `BaggagePolicy` (e.g., 7kg cabin, 23kg check-in) and `FareRules` (e.g., non-refundable vs refundable, \$50 change penalty).

Phase 5: Traveler Search, Booking & Journey Lifecycle

1. Traveler queries available flights between LHR and JFK on a given date via GET `/api/v1/flight-instances`.
2. Traveler queries lowest pricing via POST `/api/v1/fares/search-lowest-fares` in Pricing Service, which executes `getLowestFaresPerFlight`.
3. During flight operations, gate dispatchers advance flight instance status from `SCHEDULED` → `BOARDING` → `DEPARTED` → `IN_AIR` → `ARRIVED` → `COMPLETED`.

Architectural Summary: AirFlux demonstrates enterprise-grade Domain-Driven Design for aviation platforms. By enforcing strict data boundaries between route blueprints (*Flight*), seasonal recurrence rules (*FlightSchedule*), and physical daily departures (*FlightInstance*), and pairing them with commercial revenue products (*Fare*) and legal policies (*FareRules*), AirFlux eliminates data redundancy, prevents race conditions, and scales seamlessly across distributed services.